

Smart Device Programming

Laboratory 2

Dart Programming Fundamentals

Expressions, Variables, Data Types, Operators, Control Flow, Loops, Strings
and Unicode

Dart Fundamentals

A Lab-Oriented Companion for Flutter Development

Based on previous-year Lab 2 topics and exercises

B.Tech Computer Engineering

Semester V

Prepared for laboratory teaching and self-paced student practice

Contents

How to Use This Lab Manual	7
1 Dart Revision: Comments, Output, Statements and Expressions	9
1.1 Learning Objectives	9
1.2 Why Start with These Concepts?	9
1.3 Structure of a Dart Program	9
1.4 Comments in Dart	9
1.4.1 Single-Line Comments	9
1.4.2 Multi-Line Comments	10
1.4.3 Documentation Comments	10
1.5 Printing Output	10
1.6 Statements and Expressions	10
1.7 Arithmetic Expressions	11
1.8 Integer Division and Modulo	11
1.9 Order of Operations	11
1.10 Using the Math Library	11
1.11 Solved Program: Verify Sine of 45 Degrees	12
1.12 Common Mistakes	12
1.13 Mini Exercises	12
2 Variables, Identifiers, Constants and Scope	13
2.1 Learning Objectives	13
2.2 Why Variables are Needed	13
2.3 Identifiers and Naming Rules	13
2.4 Keywords in Dart	13
2.5 Declaring Variables	14
2.6 Type Inference using <code>var</code>	14
2.7 Mutable and Immutable Data	14
2.8 Difference between <code>final</code> and <code>const</code>	14
2.9 Understanding <code>final</code> and <code>const</code> Through an Analogy	15
2.10 Flutter-Oriented Example: Loading a User Profile	15
2.11 Quick Decision Guide	16
2.12 Variable Scope	16
2.13 Solved Programs	17
2.13.1 Program 1: Demonstrate Variables and Constants	17
2.13.2 Program 2: Average Rating	17
2.14 Mini Exercises from Previous Material	17
2.15 Challenges	18
2.16 Viva Questions	18

3	Data Types, Type Conversion and Operators	19
3.1	Learning Objectives	19
3.2	Meaning of Data Type	19
3.3	Number Types	19
3.3.1	int	19
3.3.2	double	19
3.3.3	num	19
3.4	String and bool	20
3.5	Object and dynamic	20
3.6	Checking Type at Runtime	20
3.7	Type Conversion	20
3.8	Parsing Strings	21
3.9	Operators with Mixed Types	21
3.10	Arithmetic, Relational and Logical Operators	21
3.11	Using <code>dart:math</code>	22
3.12	Solved Program: Teacher's Grading	22
3.13	Mini Exercises from Previous Material	23
3.14	Challenges	23
3.15	Viva Questions	23
4	Control Flow and Decision Making	25
4.1	Learning Objectives	25
4.2	What is Control Flow?	25
4.3	Boolean Values	25
4.4	Comparison Operators	25
4.5	Boolean Logic	25
4.6	String Equality	26
4.7	if Statement	26
4.8	if-else Statement	26
4.9	else-if Ladder	26
4.10	Ternary Conditional Operator	27
4.11	switch Statement	27
4.12	enum	28
4.13	Solved Programs	28
4.13.1	Program 1: Teenager Check	28
4.13.2	Program 2: Grade Calculation	28
4.14	Mini Exercises from Previous Material	29
4.15	Viva Questions	29
5	Loops and Repetition	31
5.1	Learning Objectives	31
5.2	Why Loops are Needed	31
5.3	while Loop	31
5.4	do-while Loop	31
5.5	for Loop	31
5.6	break Statement	32
5.7	continue Statement	32
5.8	Random Numbers	32
5.9	for-in Loop	33
5.10	forEach Loop	33
5.11	Solved Programs	33
5.11.1	Program 1: Factorial	33
5.11.2	Program 2: Fibonacci Number	33
5.11.3	Program 3: Next Power of Two	34

5.12	Mini Exercises from Previous Material	34
5.13	Challenges from Previous Material	34
5.14	Viva Questions	34
6	Flutter Strings, Unicode and Character Encoding	37
6.1	Learning Objectives	37
6.2	Connecting the Topic with Flutter	37
6.2.1	Displaying Text	37
6.2.2	String Interpolation in a Widget	37
6.2.3	Multilingual Text	37
6.2.4	Text Entered by a User	38
6.3	Basic Dart String Syntax	38
6.4	Dart Does Not Have a Separate <code>char</code> Type	38
6.5	From a Visible Character to Stored Data	38
6.6	Unicode, Code Points and Runes	39
6.7	Why Dart and Flutter Work with UTF-16 Strings	39
6.8	How UTF-16 Works	39
6.8.1	Basic Multilingual Plane	40
6.8.2	Characters Outside the Basic Multilingual Plane	40
6.9	Visualizing UTF-16 Code Units and Surrogate Pairs	40
6.9.1	Important Clarification	41
6.9.2	Example: One Emoji, Different Counts	41
6.10	UTF-8, UTF-16 and UTF-32	41
6.10.1	UTF-8 in Brief	42
6.10.2	UTF-32 in Brief	42
6.11	Dart Properties for Inspecting Strings	42
6.11.1	<code>codeUnits</code>	42
6.11.2	<code>runes</code>	42
6.11.3	User-Perceived Characters and Grapheme Clusters	42
6.12	Using the <code>characters</code> Package	43
6.13	Typical Flutter Text Workflow	43
6.14	Flutter-Oriented Examples	43
6.14.1	Example 1: Displaying Multilingual Text	43
6.14.2	Example 2: Validating Visible Character Length	44
6.14.3	Example 3: Safely Taking the First Visible Character	44
6.14.4	Example 4: Encoding and Decoding UTF-8 Bytes	44
6.15	Common Mistakes	44
6.16	Solved Programs	45
6.16.1	Program 1: Compare Code Units, Runes and Characters	45
6.16.2	Program 2: UTF-8 Encoding and Decoding	45
6.16.3	Program 3: Count Visible Characters	45
6.17	Exercises	46
6.18	Additional Encoding Exercises	46
6.19	Viva Questions	46
7	Programming Practice for Lab 2	49
7.1	Purpose of This Chapter	49
7.2	Program 1: Temperature Converter	49
7.3	Program 2: Simple Calculator	50
7.4	Program 3: Prime Number	50
7.5	Program 4: Armstrong Number	51
7.6	Program 5: Pattern Printing	51
7.7	Program 6: Count Vowels in a String	51
7.8	Additional Practice Programs	52

8	Lab 2 Assignment and Submission Checklist	53
8.1	Lab Assignment	53
8.2	Concept Words to Explain	53
8.3	Programs to Submit	54
8.4	Submission Format	55
8.5	Self-Assessment Checklist	55
8.6	Viva Preparation Questions	55
8.7	Outcome of Lab 2	56

How to Use This Lab Manual

This manual is designed as the second laboratory of Smart Device Programming. Lab 1 introduced Flutter, Dart, Android Studio, Flutter project creation, project structure, Hot Reload and a few very simple Dart programs. Lab 2 now focuses on the programming fundamentals that students must understand before moving into detailed Flutter user interface development.

Students are already familiar with C, C++ and Java. Therefore, this manual does not treat programming as a completely new subject. Instead, it explains Dart by connecting new syntax with familiar programming ideas. The emphasis is on writing correct Dart programs, understanding the logic, documenting the work properly and preparing for viva.

What Students Should Do

1. Read the concept explanation before directly copying programs.
2. Execute every program in Android Studio, VS Code or terminal using Dart SDK.
3. Modify the given programs and observe the output.
4. Solve all mini-exercises and challenges chapter-wise.
5. Submit the required programs with source code, output screenshots and short explanation.
6. Prepare short notes on Unicode, UTF and surrogate pairs as required in the assignment.

Lab 2 Coverage

Module	Topics Covered
Dart Revision	Comments, print, statements, expressions and program structure.
Variables and Constants	Identifiers, keywords, variables, constants, final, const, type inference and scope.
Data Types and Operators	int, double, num, bool, String, Object, dynamic, arithmetic, relational, logical, type-test, type-conversion and math library.
Control Flow	Boolean logic, if, if-else, else-if ladder, ternary operator, switch, enum and variable scope.
Loops	while, do-while, for, break, continue, random numbers, for-in and forE-ach.
Characters and Strings	String basics, codeUnits, runes, Unicode, UTF-8, UTF-16, surrogate pairs, grapheme clusters and characters package.
Programming Practice	Temperature converter, number-based programs, control-flow programs, string programs and submission exercises.

1 Dart Revision: Comments, Output, Statements and Expressions

1.1 Learning Objectives

After completing this chapter, students will be able to write simple Dart programs, document code using comments, display output using `print()`, identify statements and expressions, and use arithmetic expressions as the foundation for later programming.

1.2 Why Start with These Concepts?

Before writing larger programs, students must become comfortable with the basic building blocks of Dart. A program is not only a collection of instructions; it must also be readable, testable and easy to debug. Comments help a programmer explain the intention behind the code. The `print()` function helps display intermediate values and output. Statements tell the computer what to do, while expressions calculate values. These ideas are simple, but they appear in every Dart and Flutter program.

In C, students commonly use `printf()` for output. In C++, they use `cout`. In Java, they use `System.out.println()`. In Dart, the equivalent tool for console output is `print()`. This makes Dart programs concise and beginner-friendly.

1.3 Structure of a Dart Program

Every executable Dart program begins execution from the `main()` function. The body of the function is written inside curly braces. Each simple statement generally ends with a semicolon.

```
1 void main() {  
2     print('Hello, Dart!');  
3 }
```

The keyword `void` means that the function does not return any value. The name `main` is special because Dart starts executing the program from this function. The `print()` statement sends output to the console.

1.4 Comments in Dart

Comments are used to write explanatory text inside a program. They are ignored by the compiler. A good program should not only work correctly but should also be understandable to another programmer. Comments help with this.

1.4.1 Single-Line Comments

```
1 void main() {  
2   // This line prints a welcome message.  
3   print('Welcome to Dart Programming');  
4 }
```

1.4.2 Multi-Line Comments

```
1 void main() {  
2   /*  
3     This program demonstrates  
4     multi-line comments in Dart.  
5   */  
6   print('Comments are useful');  
7 }
```

1.4.3 Documentation Comments

Documentation comments are written using `///` or `/** ... */`. They are useful for documenting functions, classes and libraries. Flutter and Dart source code use documentation comments extensively.

```
1 /// Prints a welcome message for students.  
2 void main() {  
3   print('Welcome Students');  
4 }
```

Teaching Note

Students should be encouraged to write comments for program logic, but they should avoid writing unnecessary comments for obvious statements. For example, writing `// print name` above `print(name)` is less useful than explaining why a calculation is performed.

1.5 Printing Output

The `print()` function can display strings, numbers, variables and expressions. It is useful while learning because students can check the value of an expression immediately.

```
1 void main() {  
2   print('Hello, Dart!');  
3   print(10 + 20);  
4   print(5 * 6);  
5 }
```

Expected Output

```
Hello, Dart!  
30  
30
```

1.6 Statements and Expressions

A statement is a complete instruction given to the computer. In Dart, simple statements end with a semicolon. An expression is something that produces a value. For example, `10 + 20` is an expression because it produces the value 30. The statement `print(10 + 20);` uses that expression and displays its value.

```
1 void main() {  
2   int x = 10;      // statement  
3   int y = 20;      // statement  
4   int sum = x + y; // x + y is an expression  
5   print(sum);      // statement  
6 }
```

1.7 Arithmetic Expressions

Dart supports familiar arithmetic operators such as addition, subtraction, multiplication and division. Unlike some languages, the division operator `/` returns a decimal result.

```
1 void main() {  
2   print(2 + 6);  
3   print(10 - 2);  
4   print(2 * 4);  
5   print(24 / 3);  
6   print(22 / 7);  
7 }
```

1.8 Integer Division and Modulo

If integer division is required, Dart provides the truncating division operator `~/`. The modulo operator `%` gives the remainder.

```
1 void main() {  
2   print(22 / 7);    // normal division  
3   print(22 ~/ 7);   // integer division  
4   print(28 % 10);   // remainder  
5 }
```

1.9 Order of Operations

Dart follows standard mathematical precedence rules. Multiplication, division, integer division and modulo are evaluated before addition and subtraction. Parentheses can be used to make expressions clearer.

```
1 void main() {  
2   print(350 / 5 + 2);  
3   print(350 / (5 + 2));  
4 }
```

1.10 Using the Math Library

Dart provides a core library named `dart:math`. It contains mathematical constants and functions such as `pi`, `sin()`, `cos()`, `sqrt()`, `max()` and `min()`.

```
1 import 'dart:math';  
2  
3 void main() {  
4   print(sin(45 * pi / 180));  
5   print(cos(135 * pi / 180));  
6   print(sqrt(2));  
7   print(max(5, 10));  
8 }
```

```
8 print(min(-5, -10));
9 }
```

1.11 Solved Program: Verify Sine of 45 Degrees

Problem: Print the value of $1/\sqrt{2}$ and compare it with $\sin 45^\circ$.

```
1 import 'dart:math';
2
3 void main() {
4   double value1 = 1 / sqrt(2);
5   double value2 = sin(45 * pi / 180);
6
7   print('1 / sqrt(2) = $value1');
8   print('sin(45 degrees) = $value2');
9 }
```

Explanation: The function `sqrt(2)` calculates the square root of 2. The function `sin()` expects the angle in radians, so `45 * pi / 180` converts degrees into radians. Both expressions give approximately the same result.

1.12 Common Mistakes

Common Mistakes

- Forgetting the semicolon after a statement.
- Writing `Print()` instead of `print()`.
- Assuming `/` performs integer division.
- Forgetting to import `dart:math` before using `sqrt()` or `sin()`.
- Writing complex expressions without parentheses, making them difficult to read.

1.13 Mini Exercises

Mini Exercises

1. Print your name, department and semester using separate `print()` statements.
2. Print the values of `2 + 6`, `10 - 2`, `2 * 4` and `24 / 3`.
3. Print the value of `22 / 7` and `22 ~/ 7`. Explain the difference.
4. Print the value of `28 % 10`. Explain why the answer is 8.
5. Print the value of `1 / sqrt(2)` and confirm that it is approximately equal to `sin(45 * pi / 180)`.

2 Variables, Identifiers, Constants and Scope

2.1 Learning Objectives

After completing this chapter, students will be able to name variables correctly, identify Dart keywords, declare variables, use constants, understand `final` and `const`, apply type inference and explain variable scope.

2.2 Why Variables are Needed

A variable is a named storage location used to hold data during program execution. In real applications, values such as user name, password, marks, temperature, product price, counter value and login status must be stored temporarily. Variables make it possible to store such data, use it in expressions, modify it when required and pass it to functions.

Students familiar with C, C++ and Java already know the idea of variables. Dart keeps the same basic concept but adds modern features such as type inference using `var`, immutable values using `final` and `const`, and safer type checking.

2.3 Identifiers and Naming Rules

An identifier is the name given to a variable, function, class or other program element. Good naming is important because readable names make the program easier to understand.

- Identifiers can contain letters, digits, underscore and dollar sign.
- An identifier cannot start with a digit.
- An identifier cannot contain spaces.
- Keywords cannot be used as identifiers.
- Dart identifiers are case-sensitive.
- Meaningful names should be preferred over short unclear names.

Valid Identifiers	Invalid Identifiers
<code>firstName</code>	<code>first name</code>
<code>first_name</code>	<code>first-name</code>
<code>num1</code>	<code>1number</code>
<code>\$result</code>	<code>class</code>

2.4 Keywords in Dart

Keywords are reserved words that have special meaning in Dart. Examples include `class`, `if`, `else`, `for`, `while`, `return`, `const`, `final`, `var`, `void`, `true`, `false` and `null`. These words cannot be used as variable names.

2.5 Declaring Variables

A variable can be declared with an explicit type.

```
1 void main() {  
2   int number = 10;  
3   double price = 99.50;  
4   String name = 'Amit';  
5   bool isPassed = true;  
6  
7   print(number);  
8   print(price);  
9   print(name);  
10  print(isPassed);  
11 }
```

2.6 Type Inference using var

Dart can infer the type of a variable from the value assigned to it. This is called type inference. It reduces unnecessary typing while preserving type safety.

```
1 void main() {  
2   var city = 'Nadiad';  
3   var marks = 85;  
4   var cgpa = 8.75;  
5  
6   print(city.runtimeType);  
7   print(marks.runtimeType);  
8   print(cgpa.runtimeType);  
9 }
```

Once Dart infers a type for a `var` variable, the variable remains of that type. For example, if `marks` is inferred as `int`, assigning a string later will produce an error.

2.7 Mutable and Immutable Data

A normal variable is mutable, meaning its value can be changed after creation. Immutable data cannot be changed after it is assigned. Immutable values reduce accidental modification and make programs easier to reason about.

```
1 void main() {  
2   int age = 20;  
3   age = 21;           // allowed  
4  
5   final college = 'DDU';  
6   // college = 'Other'; // not allowed  
7  
8   const pi = 3.14159;  
9   // pi = 3.14;       // not allowed  
10 }
```

2.8 Difference between final and const

Both `final` and `const` prevent reassignment, but they are not identical. A `const` value must be known at compile time. A `final` value may be assigned at runtime but only once.

Feature	final	const
When value is fixed	Runtime or compile time	Compile time only
Can be reassigned?	No	No
Common use	Values assigned once	Constant values and constant widgets
Example	<code>final now = DateTime.now();</code>	<code>const pi = 3.14;</code>

2.9 Understanding final and const Through an Analogy

The difference between **final** and **const** becomes easier to understand when we relate it to a real-world situation. Imagine that a customer is ordering a customized T-shirt from an online store.

The standard size chart is already decided by the manufacturer. For example, the measurement for a medium-size T-shirt is known before any customer opens the website. It does not depend on who is ordering or when the order is placed. Such a value is similar to **const**, because it is known in advance and never changes.

The name printed on the T-shirt is different. The manufacturer cannot know the customer's chosen name before the order is placed. The value becomes available only when the customer enters it and submits the order. After that moment, the name for that particular shirt should not change. Such a value is similar to **final**: it is decided at runtime but can be assigned only once.

T-Shirt Analogy

- **const**: The manufacturer's fixed size chart. It is known before the order is placed.
- **final**: The customer's printed name. It is known only after the customer submits the order, but is then locked for that shirt.

```

1 void main() {
2   const mediumChestSize = 40; // known in advance
3
4   final customerName = getCustomerName(); // known at runtime
5
6   print('Medium size = $mediumChestSize inches');
7   print('Name printed on shirt = $customerName');
8 }
9
10 String getCustomerName() {
11   return 'Alex';
12 }
```

2.10 Flutter-Oriented Example: Loading a User Profile

In a Flutter application, both compile-time constants and runtime final values are common. Consider a profile screen that loads user information from a server.

The API base URL is normally fixed when the application is built. It does not depend on the currently logged-in user, so it is suitable for **const**. The user name, however, is received only after the application makes a network request. Once the response is received, the value can be stored in a **final** variable because that particular response should not be reassigned.

```

1 const apiUrl = 'https://api.example.com';
2
3 Future<void> loadProfile() async {
4   final userName = await fetchUserName();
```

```

5
6  print('$apiBaseUrl/profile');
7  print('Welcome, $userName');
8  }
9
10 Future<String> fetchUserName() async {
11     return 'Alex';
12 }

```

Why Flutter Frequently Uses const

Flutter uses many immutable widget objects. When a widget and all its arguments are known at compile time, writing `const` allows Dart and Flutter to reuse the same object instead of creating an equivalent object repeatedly. This can reduce unnecessary object creation and also clearly communicates that the widget configuration will not change.

```

1 // Flutter-style examples
2
3 const Text('Smart Device Programming');
4
5 final currentTime = DateTime.now();
6
7 final userName = await fetchUserName();

```

In the first statement, the text is known at compile time. In the second and third statements, the values become available only while the program is running. Therefore, they can be `final`, but not `const`.

2.11 Quick Decision Guide

Question	Use
Is the value completely known at compile time?	Prefer <code>const</code> .
Will the value be received or calculated at run-time but assigned only once?	Use <code>final</code> .
Will the value need to change later?	Use a normal mutable variable.
Is a Flutter widget fully fixed and all its arguments are constant?	Use a <code>const</code> constructor where available.

2.12 Variable Scope

Scope defines where a variable is visible in the program. Dart uses curly braces to define scope. A variable declared inside a block cannot be accessed outside that block.

```

1 const global = 'Hello, world';
2
3 void main() {
4     const local = 'Hello, main';
5
6     if (2 > 1) {
7         const insideIf = 'Hello, if block';
8         print(global);
9         print(local);
10        print(insideIf);
11    }

```



```
12
13 print(global);
14 print(local);
15 // print(insideIf); // Error: outside scope
16 }
```

Best Practice

Declare variables as close as possible to where they are used. This improves readability and avoids accidental modification from unrelated parts of the program.

2.13 Solved Programs

2.13.1 Program 1: Demonstrate Variables and Constants

```
1 void main() {
2   int age = 19;
3   var department = 'Computer Engineering';
4   final university = 'Dharmsinh Desai University';
5   const pi = 3.14159;
6
7   print('Age = $age');
8   print('Department = $department');
9   print('University = $university');
10  print('Pi = $pi');
11 }
```

2.13.2 Program 2: Average Rating

```
1 void main() {
2   const double rating1 = 4.5;
3   const double rating2 = 4.0;
4   const double rating3 = 5.0;
5
6   const double average = (rating1 + rating2 + rating3) / 3;
7
8   print('Average Rating = $average');
9 }
```

2.14 Mini Exercises from Previous Material

Mini Exercises

1. Declare a constant `myAge` and set it equal to your age.
2. Declare a variable `dogs` and set it equal to the number of dogs you own. Increase it by one and print the new value.
3. Create two constants `age1 = 42` and `age2 = 21`. Check that Dart infers both as `int`.
4. Create a constant `averageAge` as `(age1 + age2) / 2`. Check its type and explain why the result is `double`.
5. Declare three double constants `rating1`, `rating2` and `rating3`. Calculate their average.

2.15 Challenges

Challenges

1. **Variables:** Declare a constant integer `myAge`. Declare a variable `dogs`. Increase the number of dogs by one and print the result.
2. **Make it Compile:** Given a program where a constant is modified later, correct the program so that it compiles.
3. **Compute the Answer:** Given constants `x = 46` and `y = 10`, evaluate arithmetic expressions manually and verify using Dart.
4. **Average Rating:** Calculate the average of three ratings and print the result.
5. **Quadratic Equation:** For a quadratic equation $ax^2 + bx + c = 0$, calculate roots using the quadratic formula for suitable values of a , b and c .

2.16 Viva Questions

1. What is an identifier?
2. State any four rules for naming identifiers in Dart.
3. What is a keyword?
4. What is type inference?
5. Differentiate between `final` and `const`.
6. What is variable scope?

3 Data Types, Type Conversion and Operators

3.1 Learning Objectives

After completing this chapter, students will understand Dart data types, explicit and inferred typing, type checking, type conversion, arithmetic operators, relational operators, logical operators, mixed-type operations and useful math functions.

3.2 Meaning of Data Type

A data type tells the compiler what kind of value is stored and which operations are valid on that value. For example, addition is meaningful for numbers, but string operations are meaningful for textual data. Dart is type-safe, which means it tries to detect type-related mistakes before the program runs.

In Dart, every value is an object. The root type of most values is `Object`. Common types include `int`, `double`, `num`, `String`, `bool` and `dynamic`.

3.3 Number Types

3.3.1 `int`

The `int` type stores whole numbers.

```
1 void main() {  
2   int count = 10;  
3   print(count);  
4 }
```

3.3.2 `double`

The `double` type stores decimal numbers.

```
1 void main() {  
2   double price = 99.50;  
3   print(price);  
4 }
```

3.3.3 `num`

The `num` type can store both integers and doubles.

```
1 void main() {  
2   num value = 10;  
3   print(value);  
4  
5   value = 10.75;
```

```
6  print(value);
7  }
```

3.4 String and bool

The `String` type stores text, whereas `bool` stores `true` or `false`. Boolean values are essential in decision-making and loops.

```
1  void main() {
2    String language = 'Dart';
3    bool isUseful = true;
4
5    print(language);
6    print(isUseful);
7  }
```

3.5 Object and dynamic

The type `Object` is the parent of most Dart types. The type `dynamic` disables many compile-time type checks. Although `dynamic` is flexible, it should be used carefully because it can hide errors until runtime.

```
1  void main() {
2    dynamic value = 10;
3    print(value);
4
5    value = 'Dart';
6    print(value);
7
8    Object item = 3.14;
9    print(item.runtimeType);
10 }
```

3.6 Checking Type at Runtime

Dart provides the `is` keyword and `runtimeType` property to check the type.

```
1  void main() {
2    num myNumber = 3.14;
3
4    print(myNumber is double);
5    print(myNumber is int);
6    print(myNumber.runtimeType);
7  }
```

3.7 Type Conversion

Dart does not silently convert one type into another when precision may be lost. The programmer must explicitly state the conversion.

```
1  void main() {
2    double decimal = 12.5;
3    int integer = decimal.toInt();
4
5    print(integer);
6  }
```

```

6
7  int number = 3;
8  double converted = number.toDouble();
9
10 print(converted);
11 }

```

Precision Warning

When `12.5.toInt()` is used, the decimal part is lost and the value becomes 12. Therefore, type conversion must be used carefully.

3.8 Parsing Strings

Input values are often received as strings. To use them as numbers, parsing is required.

```

1 void main() {
2   String input1 = '100';
3   String input2 = '25.5';
4
5   int value1 = int.parse(input1);
6   double value2 = double.parse(input2);
7
8   print(value1 + 50);
9   print(value2 + 10);
10 }

```

3.9 Operators with Mixed Types

When an `int` and a `double` are used together in an arithmetic expression, Dart usually produces a `double` result to avoid loss of precision.

```

1 void main() {
2   const hourlyRate = 19.5;
3   const hoursWorked = 10;
4
5   const totalCost = hourlyRate * hoursWorked;
6
7   print(totalCost);
8   print(totalCost.runtimeType);
9 }

```

3.10 Arithmetic, Relational and Logical Operators

Category	Operators
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>~</code> , <code>%</code>
Relational	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>
Logical	<code>&&</code> , <code> </code> , <code>!</code>
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>
Type Test	<code>is</code> , <code>is!</code>
Conditional	<code>condition ? value1 : value2</code>

```
1 void main() {  
2   int a = 17;  
3   int b = 5;  
4  
5   print(a + b);  
6   print(a / b);  
7   print(a ~/ b);  
8   print(a % b);  
9   print(a > b);  
10  print(a == b);  
11 }
```

3.11 Using dart:math

The `dart:math` library provides useful mathematical functions.

```
1 import 'dart:math';  
2  
3 void main() {  
4   print(sqrt(25));  
5   print(pow(2, 5));  
6   print(max(20, 15));  
7   print(min(20, 15));  
8 }
```

3.12 Solved Program: Teacher's Grading

Problem: Attendance is worth 20%, homework is worth 30% and exam is worth 50%. A student got 90 in attendance, 80 in homework and 94 in exam. Calculate the weighted grade.

```
1 void main() {  
2   const attendance = 90;  
3   const homework = 80;  
4   const exam = 94;  
5  
6   const finalGrade = attendance * 0.20 + homework * 0.30 + exam * 0.50;  
7  
8   print('Final Grade = $finalGrade');  
9 }
```

3.13 Mini Exercises from Previous Material

Mini Exercises

1. Create constants `age1 = 42` and `age2 = 21`. Confirm the inferred type.
2. Create `averageAge = (age1 + age2) / 2`. Explain why the result is `double`.
3. Create constants `firstName` and `lastName`. Create `fullName` by joining them with a space.
4. Create a constant `myDetails` using string interpolation.
5. Convert a `double` into `int` using `toInt()` and observe loss of precision.
6. Use `is` and `runtimeType` to check the type of a value at runtime.

3.14 Challenges

Challenges

1. **Teacher's Grading:** Calculate final grade using attendance, homework and exam weightage.
2. **Find the Error:** Identify and correct errors caused by assigning a value of incompatible type.
3. **What Type?** Determine the type of `10 / 2` and verify using `runtimeType`.
4. **Type Conversion:** Convert numeric strings into numbers and perform arithmetic operations.
5. **Summary Expression:** Given several constants, compute a final summary expression and explain each step.

3.15 Viva Questions

1. What is a data type?
2. Differentiate between `int`, `double` and `num`.
3. What is `dynamic`?
4. Why does Dart require explicit type conversion?
5. What is the use of `runtimeType`?
6. What is the difference between `/` and `~/`?

4 Control Flow and Decision Making

4.1 Learning Objectives

After completing this chapter, students will be able to use Boolean expressions, comparison operators, logical operators, if statements, if-else, else-if ladder, ternary conditional operator, switch statement and enum.

4.2 What is Control Flow?

Control flow decides the order in which program statements are executed. Without control flow, a program executes line by line from top to bottom. Real applications require decisions. For example, a login screen must check whether the password is correct; a marks program must decide whether a student has passed; a temperature converter must decide which conversion formula to use. Dart provides decision-making statements for such requirements.

4.3 Boolean Values

The `bool` type stores either `true` or `false`. Boolean values are the foundation of decision-making.

```
1 void main() {  
2     const yes = true;  
3     const no = false;  
4  
5     print(yes);  
6     print(no);  
7 }
```

4.4 Comparison Operators

Comparison operators compare two values and return a Boolean result.

```
1 void main() {  
2     print(1 == 2);  
3     print(1 != 2);  
4     print(1 < 2);  
5     print(2 >= 2);  
6 }
```

4.5 Boolean Logic

Logical operators combine multiple Boolean expressions. The AND operator `&&` returns true only when both conditions are true. The OR operator `||` returns true when at least one condition is true. The NOT operator `!` reverses the Boolean value.

```
1 void main() {
2   const isSunny = true;
3   const isFinished = true;
4   const willGoCycling = isSunny && isFinished;
5
6   print(willGoCycling);
7
8   const willTravel = true;
9   const canFindPhoto = false;
10  const canDraw = willTravel || canFindPhoto;
11
12  print(canDraw);
13 }
```

4.6 String Equality

Dart compares strings using `==`. This is simpler than languages where string comparison requires a method such as `equals()`.

```
1 void main() {
2   const guess = 'dog';
3   const dogEqualsCat = guess == 'cat';
4
5   print(dogEqualsCat);
6 }
```

4.7 if Statement

The `if` statement executes a block only when the condition is true.

```
1 void main() {
2   if (2 > 1) {
3     print('Yes, 2 is greater than 1.');
```

4.8 if-else Statement

```
1 void main() {
2   const animal = 'Fox';
3
4   if (animal == 'Cat' || animal == 'Dog') {
5     print('Animal is a house pet.');
```

4.9 else-if Ladder

An else-if ladder checks multiple conditions one by one. The first true condition is executed, and the remaining conditions are skipped.

```
1 void main() {
2   const trafficLight = 'yellow';
3   var command = '';
4
5   if (trafficLight == 'red') {
6     command = 'Stop';
7   } else if (trafficLight == 'yellow') {
8     command = 'Slow down';
9   } else if (trafficLight == 'green') {
10    command = 'Go';
11  } else {
12    command = 'Invalid color';
13  }
14
15  print(command);
16 }
```

4.10 Ternary Conditional Operator

The ternary operator is a compact form of a simple if-else statement.

```
1 void main() {
2   const score = 83;
3   const message = score >= 60 ? 'You passed' : 'You failed';
4
5   print(message);
6 }
```

Readability Note

Use the ternary operator only when the condition is simple. If the logic becomes complex, use a normal if-else statement for better readability.

4.11 switch Statement

A `switch` statement is useful when one variable must be compared with many possible constant values.

```
1 void main() {
2   const audioState = 'paused';
3
4   switch (audioState) {
5     case 'playing':
6       print('Audio is playing');
7       break;
8     case 'paused':
9       print('Audio is paused');
10      break;
11     case 'stopped':
12       print('Audio is stopped');
13       break;
14     default:
15       print('Unknown state');
16   }
17 }
```

4.12 enum

An `enum` defines a fixed set of named values. It improves readability and avoids invalid string values.

```
1 enum AudioState { playing, paused, stopped }
2
3 void main() {
4   const state = AudioState.paused;
5
6   switch (state) {
7     case AudioState.playing:
8       print('Playing');
9       break;
10    case AudioState.paused:
11      print('Paused');
12      break;
13    case AudioState.stopped:
14      print('Stopped');
15      break;
16  }
17 }
```

4.13 Solved Programs

4.13.1 Program 1: Teenager Check

```
1 void main() {
2   const myAge = 18;
3
4   if (myAge >= 13 && myAge <= 19) {
5     print('Teenager');
6   } else {
7     print('Not a teenager');
8   }
9 }
```

4.13.2 Program 2: Grade Calculation

```
1 void main() {
2   int marks = 82;
3
4   if (marks >= 90) {
5     print('Grade A+');
6   } else if (marks >= 80) {
7     print('Grade A');
8   } else if (marks >= 70) {
9     print('Grade B');
10  } else if (marks >= 40) {
11    print('Pass');
12  } else {
13    print('Fail');
14  }
15 }
```

4.14 Mini Exercises from Previous Material

Mini Exercises

1. Create a constant `myAge`. Create `isTeenager` using Boolean logic to check whether the age is between 13 and 19.
2. Create `maryAge = 30`. Create `bothTeenagers` to check whether both you and Mary are teenagers.
3. Create string constants `reader` and `ray`. Create a Boolean `rayIsReader` using string equality.
4. Write an if statement to print `Teenager` or `Not a teenager`.
5. Replace the previous if-else with a ternary conditional operator and store the result in `answer`.
6. Make an enum `AudioState` with values `playing`, `paused` and `stopped`.

4.15 Viva Questions

1. What is control flow?
2. What is a Boolean value?
3. Differentiate between `==` and `=`.
4. What is the difference between `&&` and `||`?
5. What is the ternary conditional operator?
6. When should switch be preferred over else-if?
7. What is enum?

5 Loops and Repetition

5.1 Learning Objectives

After completing this chapter, students will be able to use while, do-while, for, break, continue, random numbers, for-in and forEach loops in Dart.

5.2 Why Loops are Needed

Many programming tasks require repetition. A program may need to print numbers from 1 to 100, process all marks in a list, repeat input until valid data is entered or continue a game until the user exits. Writing the same statement again and again is inefficient and error-prone. Loops solve this problem by executing a block repeatedly while a condition remains true.

5.3 while Loop

A **while** loop checks the condition before each iteration. If the condition is false initially, the body does not execute even once.

```
1 void main() {  
2   var sum = 1;  
3  
4   while (sum < 10) {  
5     sum += 4;  
6     print(sum);  
7   }  
8 }
```

5.4 do-while Loop

A **do-while** loop checks the condition after executing the body. Therefore, the body executes at least once.

```
1 void main() {  
2   var sum = 11;  
3  
4   do {  
5     sum += 4;  
6   } while (sum < 10);  
7  
8   print(sum);  
9 }
```

5.5 for Loop

A **for** loop is commonly used when the number of iterations is known.

```
1 void main() {  
2   for (var i = 0; i < 5; i++) {  
3     print(i);  
4   }  
5 }
```

5.6 break Statement

The `break` statement stops the loop immediately.

```
1 void main() {  
2   var sum = 1;  
3  
4   while (true) {  
5     sum += 4;  
6  
7     if (sum > 10) {  
8       break;  
9     }  
10  }  
11  
12  print(sum);  
13 }
```

5.7 continue Statement

The `continue` statement skips the remaining part of the current iteration and moves to the next iteration.

```
1 void main() {  
2   for (var i = 0; i < 5; i++) {  
3     if (i == 2) {  
4       continue;  
5     }  
6     print(i);  
7   }  
8 }
```

5.8 Random Numbers

Dart can generate random numbers using `Random` from `dart:math`. This is useful in games, simulations and testing.

```
1 import 'dart:math';  
2  
3 void main() {  
4   final random = Random();  
5  
6   while (random.nextInt(6) + 1 != 6) {  
7     print('Not a six!');  
8   }  
9  
10  print('Finally, you got a six!');  
11 }
```


5.9 for-in Loop

A for-in loop is useful when iterating over a collection.

```
1 void main() {  
2   const myString = 'I love Dart';  
3  
4   for (var codePoint in myString.runes) {  
5     print(String.fromCharCode(codePoint));  
6   }  
7 }
```

5.10 forEach Loop

Collections also provide a forEach method.

```
1 void main() {  
2   const numbers = [1, 2, 3];  
3  
4   numbers.forEach((number) {  
5     print(number);  
6   });  
7 }
```

5.11 Solved Programs

5.11.1 Program 1: Factorial

```
1 void main() {  
2   int n = 5;  
3   int fact = 1;  
4  
5   for (int i = 1; i <= n; i++) {  
6     fact *= i;  
7   }  
8  
9   print('Factorial = $fact');  
10 }
```

5.11.2 Program 2: Fibonacci Number

```
1 void main() {  
2   int n = 8;  
3   int first = 1;  
4   int second = 1;  
5  
6   if (n == 1 || n == 2) {  
7     print(1);  
8   } else {  
9     int current = 0;  
10  
11     for (int i = 3; i <= n; i++) {  
12       current = first + second;  
13       first = second;  
14       second = current;  
15     }  
16   }
```

```
16
17     print('Fibonacci number = $current');
18 }
19 }
```

5.11.3 Program 3: Next Power of Two

```
1 void main() {
2     int number = 37;
3     int power = 1;
4
5     while (power < number) {
6         power *= 2;
7     }
8
9     print('Next power of two = $power');
10 }
```

5.12 Mini Exercises from Previous Material

Mini Exercises

1. Create a variable `counter = 0`. Use a while loop with condition `counter < 10`. Print `counter` is X and increment counter.
2. Write a for loop from 1 to 10 inclusive and print the square of each number.
3. Write a for-in loop over [1, 2, 4, 7] and print the square root of each number.
4. Repeat the previous exercise using `forEach`.

5.13 Challenges from Previous Material

Challenges

1. **Find the Error:** Analyze the code where `lastName` is declared inside if and else-if blocks. Explain the scope problem.
2. **Boolean Challenge:** Evaluate given Boolean expressions manually and verify using Dart.
3. **Next Power of Two:** Given a number, determine the next power of two above or equal to that number.
4. **Fibonacci:** Calculate the nth Fibonacci number.
5. **How Many Times?** Given a for loop, determine final sum and number of iterations.
6. **Final Countdown:** Print countdown from 10 to 0.
7. **Print Sequence:** Print 0.0, 0.1, 0.2, ..., 1.0.

5.14 Viva Questions

1. What is a loop?

2. Difference between while and do-while.
3. What is an infinite loop?
4. Difference between break and continue.
5. What is for-in loop?
6. What is forEach?

6 Flutter Strings, Unicode and Character Encoding

6.1 Learning Objectives

After completing this chapter, students will be able to use Dart strings in Flutter-style code, explain why Dart strings are based on UTF-16 code units, distinguish among UTF-8, UTF-16 and UTF-32, use `codeUnits`, `runes` and the `characters` package, and solve string and Unicode-based exercises.

6.2 Connecting the Topic with Flutter

Before studying encoding, it is important to first see where strings appear in Flutter applications. Flutter uses Dart, and therefore every text value passed to a Flutter widget is a Dart **String**. Button captions, labels, form input, validation messages, API responses and multilingual text are all represented using strings.

6.2.1 Displaying Text

```
1 // Flutter code snippet
2
3 Text('Welcome to Flutter')
```

6.2.2 String Interpolation in a Widget

```
1 String userName = 'Riya';
2
3 Text('Welcome, $userName')
```

6.2.3 Multilingual Text

```
1 Column(
2   children: const [
3     Text('Hello'),
4     Text('\u{0AA8}\u{0AAE}\u{0AB8}\u{0ACD}\u{0AA4}\u{0AC7}'),
5     Text('\u{0939}\u{093F}\u{0928}\u{094D}\u{0926}\u{0940}'),
6     Text('\u{1F600}'),
7   ],
8 )
```

The same `Text` widget can display English, Gujarati, Hindi, symbols and emojis because Dart strings support Unicode.

6.2.4 Text Entered by a User

```
1 // Flutter code snippet
2
3 final controller = TextEditingController();
4
5 TextField(
6   controller: controller,
7 )
8
9 final enteredText = controller.text;
```

The value returned by `controller.text` is a Dart **String**. This is why understanding strings and character representation is directly useful in Flutter.

6.3 Basic Dart String Syntax

A Dart **String** stores textual data. Dart supports single quotes, double quotes, multi-line strings, escape sequences, raw strings and interpolation.

```
1 void main() {
2   String language = 'Dart';
3   String framework = "Flutter";
4
5   print('$language is used with $framework.');
```

```
6   print('First line\nSecond line');
7   print(r'C:\flutter\bin');
8 }
```

6.4 Dart Does Not Have a Separate char Type

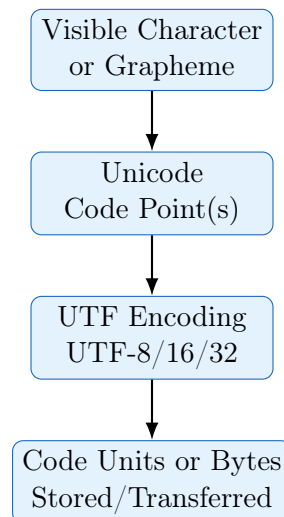
In C, C++ and Java, students may have used a **char** type. Dart does not provide a separate character type. Even one visible letter is represented as a **String**.

```
1 void main() {
2   const letter = 'A';
3
4   print(letter.runtimeType);
5   print(letter.length);
6 }
```

This works well for ordinary letters, but some visible characters such as emojis are more complex. To understand why, we must distinguish among characters, Unicode code points and encoding units.

6.5 From a Visible Character to Stored Data

When a user sees a character on the screen, several layers are involved:



Unicode defines the number assigned to a symbol. UTF encodings define how that Unicode value is represented as code units or bytes.

6.6 Unicode, Code Points and Runes

Unicode is a universal standard that assigns a unique numeric value to characters from many languages and symbol systems. Each assigned value is called a **code point**. In Dart, a Unicode code point is commonly called a **runes**.

For example, the letter A has Unicode code point U+0041. An emoji such as the dartboard has a much larger code point.

```
1 void main() {  
2   const letter = 'A';  
3   const dartboard = '\u{1F3AF}';  
4  
5   print(letter.runes.toList());  
6   print(dartboard.runes.toList());  
7 }
```

6.7 Why Dart and Flutter Work with UTF-16 Strings

Flutter does not define a separate string representation. Flutter uses Dart strings, and a Dart **String** is a sequence of UTF-16 code units. Therefore, whenever Flutter receives a string for a **Text** widget, text field, API model or validation message, that string follows Dart's UTF-16-based representation.

UTF-16 uses 16-bit code units. Many commonly used Unicode characters can be represented using one 16-bit code unit. Characters outside the Basic Multilingual Plane require two code units, known as a surrogate pair.

Important Distinction

Dart stores strings as UTF-16 code units, but files and network data are often encoded using UTF-8. A Flutter application may therefore receive UTF-8 bytes from a server, decode them into a Dart UTF-16 string, and then display that string through a widget.

6.8 How UTF-16 Works

6.8.1 Basic Multilingual Plane

Unicode code points from U+0000 to U+FFFF belong mainly to the Basic Multilingual Plane. Most common scripts can be represented by one UTF-16 code unit, except the reserved surrogate range.

```
1 void main() {
2   const text = 'A';
3
4   print(text.codeUnits);
5   print(text.runes.toList());
6 }
```

For A, both lists contain 65 because the character needs only one UTF-16 code unit.

6.8.2 Characters Outside the Basic Multilingual Plane

Some symbols and emojis have code points above U+FFFF. One 16-bit code unit is insufficient to represent them. UTF-16 therefore uses two code units:

- a high surrogate;
- a low surrogate.

Together, these two code units represent one Unicode code point.

```
1 void main() {
2   const dartboard = '\u{1F3AF}';
3
4   print(dartboard);
5   print(dartboard.length);
6   print(dartboard.codeUnits);
7   print(dartboard.runes.length);
8 }
```

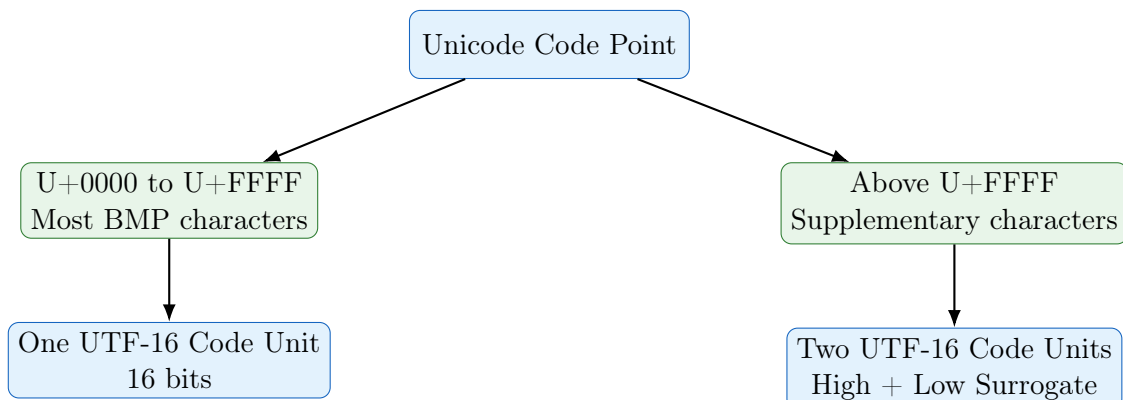
The visible dartboard is one Unicode code point, but its Dart string length is 2 because `length` counts UTF-16 code units.

Surrogate Pair

A surrogate pair is not two visible characters. It is a pair of UTF-16 code units used together to represent one Unicode code point above U+FFFF.

6.9 Visualizing UTF-16 Code Units and Surrogate Pairs

The following diagram summarizes how UTF-16 represents Unicode values. A character whose code point lies within the Basic Multilingual Plane normally uses one 16-bit code unit. A character above U+FFFF requires two 16-bit code units: one high surrogate and one low surrogate.



6.9.1 Important Clarification

A surrogate pair consists of two UTF-16 code units, but together they represent one Unicode code point. This is different from a grapheme cluster. A grapheme cluster is one user-perceived character on the screen and may contain one code point or many code points.

For example:

- A basic emoji such as a rocket may be one Unicode code point, represented by a UTF-16 surrogate pair.
- A flag emoji is usually made from two regional-indicator code points.
- A family or couple emoji may combine several emoji code points, skin-tone modifiers and zero-width joiners into one visible grapheme cluster.

Three Different Counts

For the same displayed text:

- `String.length` counts UTF-16 code units.
- `runes.length` counts Unicode code points.
- `characters.length` counts user-perceived characters or grapheme clusters.

6.9.2 Example: One Emoji, Different Counts

```
1 import 'package:characters/characters.dart';
2
3 void main() {
4   const rocket = '\u{1F680}';
5
6   print('Visible text = $rocket');
7   print('UTF-16 code units = ${rocket.length}');
8   print('Unicode code points = ${rocket.runes.length}');
9   print('Visible characters = ${rocket.characters.length}');
10 }
```

For the rocket emoji, the string usually contains two UTF-16 code units, one Unicode code point and one visible character.

6.10 UTF-8, UTF-16 and UTF-32

All three are Unicode Transformation Formats. They represent the same Unicode code points but use different code-unit sizes and storage strategies.

Encoding	Code-Unit Size	Main Characteristics
UTF-8	8 bits	Uses 1–4 bytes. ASCII text requires one byte per character. Widely used for files, web pages and network communication.
UTF-16	16 bits	Uses one or two 16-bit code units. Dart strings are sequences of UTF-16 code units. Characters above U+FFFF use surrogate pairs.
UTF-32	32 bits	Uses one 32-bit code unit for each Unicode code point. Simple to index by code point but generally consumes more memory.

6.10.1 UTF-8 in Brief

UTF-8 is variable length. ASCII characters use one byte, while other characters use two, three or four bytes. The first byte indicates the length of the sequence, and continuation bytes begin with the binary prefix 10.

Bytes	General Bit Pattern
1	0xxxxxxx
2	110xxxxx 10xxxxxx
3	1110xxxx 10xxxxxx 10xxxxxx
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

UTF-8 is commonly preferred for files and internet communication because English text is compact and ASCII-compatible.

6.10.2 UTF-32 in Brief

UTF-32 uses one 32-bit code unit for each Unicode code point. Therefore, one code point always corresponds to one UTF-32 code unit. However, a user-perceived character may still contain multiple Unicode code points, so UTF-32 does not guarantee that one visible character equals one code point. UTF-32 is simple for code-point indexing but generally consumes more memory.

6.11 Dart Properties for Inspecting Strings

6.11.1 codeUnits

The `codeUnits` property returns the UTF-16 code units.

```
1 void main() {  
2   const message = 'Hello!';  
3  
4   print(message.codeUnits);  
5 }
```

6.11.2 runes

The `runes` property returns Unicode code points.

```
1 void main() {  
2   const symbol = '\u{1F3AF}';  
3  
4   print(symbol.codeUnits);  
5   print(symbol.runes.toList());  
6 }
```

6.11.3 User-Perceived Characters and Grapheme Clusters

A user-perceived character may contain one code point or several code points. For example, a family emoji can combine multiple people emojis using zero-width joiners. Such a sequence appears as one visible symbol but consists of several runes and many UTF-16 code units.

For user-facing operations such as counting visible characters, cursor movement or truncating text safely, use the `characters` package.

6.12 Using the characters Package

In a Dart project, run:

```
dart pub add characters
```

In a Flutter project, run:

```
flutter pub add characters
```

Then import the package:

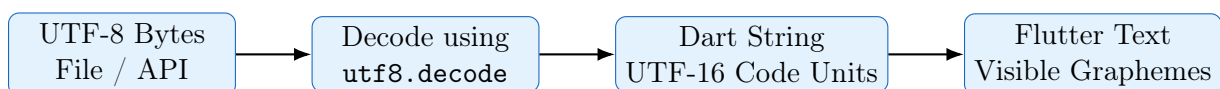
```
1 import 'package:characters/characters.dart';
2
3 void main() {
4   const family =
5     '\u{1F468}\u{200D}\u{1F469}\u{200D}\u{1F467}\u{200D}\u{1F466}';
6
7   print('UTF-16 code units = ${family.length}');
8   print('Unicode code points = ${family.runes.length}');
9   print('Visible characters = ${family.characters.length}');
10 }
```

Flutter-Specific Recommendation

When validating a user's visible input length, especially for names, emojis or multilingual text, `characters.length` is usually more meaningful than `String.length`. The latter counts UTF-16 code units, not user-perceived characters.

6.13 Typical Flutter Text Workflow

A Flutter application often receives UTF-8 bytes from a file or web service. Dart decodes those bytes into a `String`, which is then handled as UTF-16 code units. Flutter finally renders the user-perceived characters on the screen.



6.14 Flutter-Oriented Examples

6.14.1 Example 1: Displaying Multilingual Text

```
1 // Flutter code snippet
2
3 const Column(
4   children: [
5     Text('English'),
6     Text('\u{0A97}\u{0AC1}\u{0A9C}\u{0AB0}\u{0ABE}\u{0AA4}\u{0AC0}'),
7     Text('\u{0939}\u{093F}\u{0928}\u{094D}\u{0926}\u{0940}'),
8     Text('\u{1F600}'),
9   ],
10 )
```

6.14.2 Example 2: Validating Visible Character Length

```
1 import 'package:characters/characters.dart';
2
3 String? validateDisplayName(String value) {
4   if (value.characters.length < 3) {
5     return 'Enter at least 3 visible characters.';
6   }
7
8   return null;
9 }
```

6.14.3 Example 3: Safely Taking the First Visible Character

```
1 import 'package:characters/characters.dart';
2
3 void main() {
4   const text = '\u{1F468}\u{200D}\u{1F469}\u{200D}\u{1F467}\u{200D}\u{1F466} Family';
5
6   final firstVisibleCharacter = text.characters.first;
7
8   print(firstVisibleCharacter);
9 }
```

6.14.4 Example 4: Encoding and Decoding UTF-8 Bytes

Dart strings use UTF-16 code units internally, while network and file content often uses UTF-8. The `dart:convert` library performs the conversion.

```
1 import 'dart:convert';
2
3 void main() {
4   const text = 'Flutter \u{1F600}';
5
6   final utf8Bytes = utf8.encode(text);
7   final decodedText = utf8.decode(utf8Bytes);
8
9   print(utf8Bytes);
10  print(decodedText);
11 }
```

6.15 Common Mistakes

Common Mistakes

- Assuming that `String.length` always equals the number of visible characters.
- Treating one UTF-16 code unit as one complete character.
- Reversing complex emoji strings using `split('')` and expecting correct results.
- Confusing Unicode code points with UTF-8 bytes.
- Saying that Flutter independently chooses UTF-16; Flutter uses Dart strings, and Dart strings are based on UTF-16 code units.

6.16 Solved Programs

6.16.1 Program 1: Compare Code Units, Runes and Characters

```
1 import 'package:characters/characters.dart';
2
3 void main() {
4   const text = '\u{1F600}';
5
6   print('String = $text');
7   print('codeUnits = ${text.codeUnits}');
8   print('runes = ${text.runes.toList()}');
9   print('String.length = ${text.length}');
10  print('runes.length = ${text.runes.length}');
11  print('characters.length = ${text.characters.length}');
12 }
```

6.16.2 Program 2: UTF-8 Encoding and Decoding

```
1 import 'dart:convert';
2
3 void main() {
4   const original = 'Dart \u{1F3AF}';
5
6   final bytes = utf8.encode(original);
7   final restored = utf8.decode(bytes);
8
9   print('Original = $original');
10  print('UTF-8 bytes = $bytes');
11  print('Decoded = $restored');
12 }
```

6.16.3 Program 3: Count Visible Characters

```
1 import 'package:characters/characters.dart';
2
3 void main() {
4   const text =
5     '\u{1F468}\u{200D}\u{1F469}\u{200D}\u{1F467}\u{200D}\u{1F466}';
6
7   print('Visible characters = ${text.characters.length}');
8 }
```

6.17 Exercises

String, Unicode and Encoding Exercises

1. Write a Flutter-style `Text` snippet that displays English, Gujarati, Hindi and one emoji.
2. Write a Dart program to demonstrate string interpolation.
3. Print the UTF-16 code units of `Hello!`.
4. Print the runes of the dartboard emoji.
5. Compare `length`, `runes.length` and `characters.length` for a complex emoji.
6. Encode a multilingual string using UTF-8 and decode it again.
7. Explain why `String.length` may be 2 for one emoji.
8. Explain the difference among Unicode, code point, code unit and grapheme cluster.
9. Compare UTF-8, UTF-16 and UTF-32 in terms of unit size, variable length and common use.
10. Write a Flutter form validator that counts visible characters using the `characters` package.
11. Write notes on how Dart works with characters.
12. Explain surrogate pairs with a suitable example.

6.18 Additional Encoding Exercises

Encoding-Focused Practice

1. Explain why UTF-16 is called a variable-length encoding.
2. Give one example where one Unicode code point uses one UTF-16 code unit.
3. Give one example where one Unicode code point uses two UTF-16 code units.
4. Explain why a visible emoji may contain several Unicode code points.
5. Compare the storage of the English letter `A`, the rupee symbol and a rocket emoji in UTF-8.
6. Explain why UTF-32 is fixed-length at the code-point level but not necessarily at the grapheme-cluster level.
7. Write a program that prints `length`, `runes.length` and `characters.length` for a family emoji.

6.19 Viva Questions

1. Does Dart have a separate `char` type?
2. What is Unicode?
3. What is a Unicode code point?

4. What is a rune in Dart?
5. What is a code unit?
6. Why is a Dart string described as a sequence of UTF-16 code units?
7. What is the Basic Multilingual Plane?
8. What is a surrogate pair?
9. How does UTF-8 differ from UTF-16?
10. Why does UTF-32 usually consume more memory?
11. What is a grapheme cluster?
12. Why can `String.length` differ from the number of visible characters?
13. What is the use of the `characters` package?
14. How are UTF-8 bytes converted into a Dart string?
15. Why is this topic important for Flutter forms and multilingual applications?
16. Why is UTF-16 described as variable length?
17. How many UTF-16 code units are used for a BMP character?
18. How many UTF-16 code units are used for a supplementary-plane code point?
19. What is the difference between a surrogate pair and a grapheme cluster?
20. Why can one visible emoji contain many code points?
21. Does UTF-32 guarantee one visible character per code unit? Explain.

7 Programming Practice for Lab 2

7.1 Purpose of This Chapter

This chapter gives complete programs that students should practice after understanding the concepts. Each program reinforces variables, input/output, operators, control flow, loops and strings. Students should type the program themselves, run it, modify it and attach output screenshots in their lab submission.

7.2 Program 1: Temperature Converter

Aim: To convert Celsius to Fahrenheit and Fahrenheit to Celsius using menu selection and switch statement.

```
1 import 'dart:io';
2
3 void main() {
4   print('A: Convert Celsius to Fahrenheit');
5   print('B: Convert Fahrenheit to Celsius');
6
7   String selection;
8
9   do {
10    print('Enter your choice A or B:');
11    selection = stdin.readLineSync()!.toUpperCase();
12  } while (selection != 'A' && selection != 'B');
13
14  print('Enter the starting temperature:');
15  String inputTemp = stdin.readLineSync()!;
16  double temp = double.parse(inputTemp);
17
18  switch (selection) {
19    case 'A':
20      double fahrenheit = temp * 1.8 + 32;
21      print('$temp degrees Celsius is $fahrenheit degrees Fahrenheit');
22      break;
23
24    case 'B':
25      double celsius = (temp - 32) / 1.8;
26      print('$temp degrees Fahrenheit is $celsius degrees Celsius');
27      break;
28  }
29 }
```

Explanation: The program first asks the user to choose the conversion type. The do-while loop ensures that the user enters only A or B. The temperature is read as a string and converted into a double using `double.parse()`. The switch statement selects the correct formula.

7.3 Program 2: Simple Calculator

```
1 import 'dart:io';
2
3 void main() {
4   print('Enter first number:');
5   double a = double.parse(stdin.readLineSync()!);
6
7   print('Enter second number:');
8   double b = double.parse(stdin.readLineSync()!);
9
10  print('Enter operator (+, -, *, /):');
11  String op = stdin.readLineSync()!;
12
13  switch (op) {
14    case '+':
15      print('Result = ${a + b}');
16      break;
17    case '-':
18      print('Result = ${a - b}');
19      break;
20    case '*':
21      print('Result = ${a * b}');
22      break;
23    case '/':
24      if (b != 0) {
25        print('Result = ${a / b}');
26      } else {
27        print('Division by zero is not allowed');
28      }
29      break;
30    default:
31      print('Invalid operator');
32  }
33 }
```

7.4 Program 3: Prime Number

```
1 import 'dart:io';
2
3 void main() {
4   print('Enter number:');
5   int n = int.parse(stdin.readLineSync()!);
6
7   bool isPrime = true;
8
9   if (n <= 1) {
10     isPrime = false;
11   } else {
12     for (int i = 2; i <= n ~/ 2; i++) {
13       if (n % i == 0) {
14         isPrime = false;
15         break;
16       }
17     }
18   }
19
20   print(isPrime ? 'Prime' : 'Not Prime');
```

```
21 }
```

7.5 Program 4: Armstrong Number

```
1 import 'dart:io';
2 import 'dart:math';
3
4 void main() {
5   print('Enter number:');
6   int number = int.parse(stdin.readLineSync());
7
8   int temp = number;
9   int digits = number.toString().length;
10  int sum = 0;
11
12  while (temp > 0) {
13    int digit = temp % 10;
14    sum += pow(digit, digits).toInt();
15    temp ~/= 10;
16  }
17
18  if (sum == number) {
19    print('Armstrong Number');
20  } else {
21    print('Not Armstrong Number');
22  }
23 }
```

7.6 Program 5: Pattern Printing

```
1 void main() {
2   int rows = 5;
3
4   for (int i = 1; i <= rows; i++) {
5     String line = '';
6
7     for (int j = 1; j <= i; j++) {
8       line += '* ';
9     }
10
11    print(line);
12  }
13 }
```

7.7 Program 6: Count Vowels in a String

```
1 void main() {
2   String text = 'Dart Programming';
3   int count = 0;
4
5   for (int i = 0; i < text.length; i++) {
6     String ch = text[i].toLowerCase();
7
8     if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
```

```
9     count++;  
10    }  
11  }  
12  
13  print('Number of vowels = $count');  
14 }
```

7.8 Additional Practice Programs

Additional Practice

1. Find largest of three numbers.
2. Check whether a year is leap year.
3. Calculate electricity bill using slab rates.
4. Calculate simple and compound interest.
5. Print multiplication table of a number.
6. Reverse a number.
7. Check whether a number is palindrome.
8. Print all prime numbers between 1 and 100.
9. Count digits of a number.
10. Count words in a sentence.
11. Replace a word in a string.
12. Create a menu-driven unit converter.

8 Lab 2 Assignment and Submission Checklist

8.1 Lab Assignment

Students are expected to complete the following activities:

1. Explain the important concepts introduced in this laboratory with suitable examples wherever applicable.
2. Solve and execute all the mini exercises and challenge programs provided in this laboratory.
3. Study the concepts of Unicode, UTF encodings, surrogate pairs, and Dart string handling, and prepare short notes in your own words.
4. Prepare and submit a single PDF containing all the required programs in the prescribed submission format.

8.2 Concept Words to Explain

Students should prepare short explanations with examples for the following terms.

- | | |
|--------------------------|----------------------|
| 1. Comment | 15. Runtime type |
| 2. Documentation comment | 16. int |
| 3. print | 17. double |
| 4. Statement | 18. num |
| 5. Expression | 19. String |
| 6. Operator | 20. bool |
| 7. Operand | 21. dynamic |
| 8. Identifier | 22. Object |
| 9. Keyword | 23. Type conversion |
| 10. Variable | 24. Type casting |
| 11. Constant | 25. Boolean logic |
| 12. final | 26. Scope |
| 13. const | 27. if statement |
| 14. Type inference | 28. switch statement |
| | 29. enum |

- | | |
|-------------------|----------------------|
| 30. while loop | 37. Unicode |
| 31. do-while loop | 38. UTF-8 |
| 32. for loop | 39. UTF-16 |
| 33. break | 40. Code unit |
| 34. continue | 41. Rune |
| 35. for-in loop | 42. Surrogate pair |
| 36. forEach | 43. Grapheme cluster |

8.3 Programs to Submit

1. Program demonstrating comments and print statements.
2. Program demonstrating statements and expressions.
3. Program using `dart:math: sqrt, sin, cos, max, min`.
4. Program demonstrating valid variables and constants.
5. Program for average rating.
6. Program for quadratic equation roots.
7. Program demonstrating type inference and runtime type checking.
8. Program demonstrating type conversion.
9. Program for teacher's grading.
10. Program using Boolean logic to check teenager condition.
11. Program using if-else and ternary operator.
12. Program using switch and enum.
13. Program using while loop.
14. Program using do-while loop.
15. Program using for loop.
16. Program demonstrating break and continue.
17. Program for next power of two.
18. Program for Fibonacci number.
19. Program for countdown from 10 to 0.
20. Program for temperature conversion.
21. Program showing codeUnits and runes of strings.
22. Program demonstrating Unicode escape sequences.
23. Program explaining surrogate pair behavior.
24. Program using the characters package for grapheme clusters.

8.4 Submission Format

For every program, students should submit:

1. Program title.
2. Source code with proper comments.
3. Output screenshot.
4. Short explanation of logic.
5. Any observation or difficulty faced.

8.5 Self-Assessment Checklist

Before Submission, Confirm the Following

1. All programs compile successfully.
2. Output screenshots are attached.
3. Programs are properly formatted.
4. Variables have meaningful names.
5. Concepts are explained in your own words.
6. Mini-exercises are solved chapter-wise.
7. Challenge exercises are attempted.
8. Unicode, UTF and surrogate pair notes are prepared.
9. Temperature converter program is executed successfully.

8.6 Viva Preparation Questions

1. Why does Dart not have a separate char type?
2. What is the difference between codeUnits and runes?
3. Why does UTF-8 use multiple bytes?
4. What is the purpose of continuation bytes in UTF-8?
5. Why are surrogate pairs required in UTF-16?
6. What is the difference between visible character count and string length?
7. Why is type inference useful?
8. Why does Dart avoid implicit type conversion?
9. How is switch different from if-else ladder?
10. Why is do-while guaranteed to execute at least once?

8.7 Outcome of Lab 2

After completing Lab 2, students should be able to write basic Dart programs confidently, use variables and operators correctly, control program flow using conditions and loops, work with strings, understand Unicode representation and prepare for the next laboratory involving functions and collections.